

Rapport de projet MIND

Intégration de données médicales hétérogènes

Master MIND

GOURDETTE Beeverly
MNEMOI Lina

Année universitaire 2025-2026

Table des matières

1 Introduction **4**

2 Problème : description du projet et enjeux **4**
2.1 Besoins 4
2.2 Pipeline du projet 4

3 Analyse des données **5**
3.1 Présentation des données 5
3.2 Hétérogénéité des données 5

4 Méthode : choix conceptuels **7**
4.1 Solutions explorées 7
4.2 Solution retenue : modèle graphe RDF 8

5 Implémentation de la solution **9**
5.1 Pipeline : Méthode de construction de parseurs 9
5.2 Extensbilité 9
5.3 Interface d'interrogation 10
5.3.1 Structure 10
5.3.2 Architecture et optimisation des performances 11

6 Limitations **11**

7 Conclusion **12**

8 Bibliographie **13**

Abstract

Le cancer colorectal constitue un enjeu majeur de santé publique, nécessitant une analyse précise des polypes détectés lors des vidéocoloscopies afin d'évaluer leur potentiel évolutif. Cependant, les données disponibles pour l'apprentissage automatique sont dispersées à travers de nombreuses bases hétérogènes, caractérisées par des différences de formats, de systèmes de labellisation et de métadonnées. Dans ce travail, nous proposons une approche d'intégration et d'unification de ces données médicales au sein d'une base de données structurée sous forme de graphe RDF. Cette représentation permet de standardiser l'accès aux données et de faciliter leur interrogation à travers le langage SPARQL. Afin de rendre l'exploitation de cette base accessible, nous avons également développé une interface permettant de générer et d'exécuter des requêtes de manière simplifiée grâce à des filtres utilisateurs. Un système d'indexation permet de réduire significativement le temps d'interrogation de la base de données. Notre approche vise à faciliter la constitution de jeux de données exploitables pour l'apprentissage automatique dans le contexte de l'analyse des polypes colorectaux. Une documentation associée assure l'extensibilité pour l'intégration de nouvelles bases de données.

1 Introduction

Le cancer colorectal est un enjeu de santé publique majeur en raison de sa prévalence et de son potentiel évolutif. Son diagnostic repose sur la vidéocoloscopie, un examen permettant d'observer la muqueuse du côlon et d'identifier la présence de polypes. Ces polypes peuvent être bénins ou évoluer vers des formes cancéreuses, ce qui rend leur caractérisation essentielle pour la prise en charge clinique.
Afin d'évaluer le niveau de risque associé à ces polypes, les practiciens utilisent des systèmes de classification médicaux comme CONECCT, Kudo ou encore Paris (en annexe pour référence). Ces systèmes permettent de décrire les polypes selon plusieurs critères visuels, morphologiques et histologiques. Ils sont essentiels pour estimer leur niveau de risque ainsi que pour guider la décision médicale, notamment la nécessité de résection.
Des recherches en cours visent à entraîner des modèles d'apprentissage profond afin de classifier les polypes, dans le but d'aider le processus de décision médicale.

2 Problème : description du projet et enjeux

Le projet s'inscrit dans un travail doctoral dont l'objectif est de concevoir une méthode hybride de classification des polypes colorectaux par apprentissage multitâche utilisant la classification CONECCT afin d'aider à mieux préduire le caractère à risque des polypes et d'assister la prise de décision (résection ou non des polypes si jugés à risque). Cependant, l'entraînement de ce type de modèle repose sur la disponibilité de données en grande quantité, mais surtout de qualité et cohérentes. Les 22 bases de données disponibles pour le projet doctoral ne le permettent pas du fait de leur forte hétérogénéité.
Cette hétérogénéité concerne à la fois :

- les formats de données et de stockage,
- les classifications et labels utilisés pour les diagnostics,
- les annotations associées aux images,
- ainsi que la présence ou l'absence de métadonnées essentielles.

L'objectif de ce projet est donc de concevoir une solution pour intégrer les divers jeux de données dans une base unifiée afin de faciliter leur accès et leur gestion à long terme dans un contexte d'apprentissage automatique et de deep learning.

La problématique qui guide notre projet est la suivante : Comment intégrer des datasets médicaux hétérogènes dans une base unifiée permettant la génération de jeux de données pour l'apprentissage automatique ?

2.1 Besoins

Afin de répondre à la problématique posée, le système doit satisfaire plusieurs besoins fonctionnels et techniques :

- Fournir une base de données unifiée permettant l'intégration de jeux de données hétérogènes.
- Permettre des requêtes multi-critères (type de diagnostic, résolution, annotations, source des données, etc.).
- Garantir la traçabilité et la structuration des données médicales et de leurs métadonnées.
- Faciliter la génération de jeux de données pour l'apprentissage automatique.
- Assurer une évolutivité permettant l'ajout de nouvelles bases de données.

2.2 Pipeline du projet

Le pipeline du projet s'articule autour de plusieurs étapes successives permettant l'intégration des données :

1. Analyse des données : étude des structures, formats, schémas de labellisation et métadonnées propres à chaque base de données.
2. Conception du modèle de représentation : définition d'un schéma unifié permettant de représenter les images, annotations et métadonnées sous forme structurée.
3. Implémentation des parsers : développement de scripts spécifiques à chaque base afin d'extraire et normaliser les données vers un format commun (RDF/Turtle).
4. Construction de la base unifée grâce aux parsers
5. Interface d'interrogation : développement d'une interface facilitant l'accès aux données via des filtres et requêtes pré-définies
6. Documentation et extensibilité : rédaction d'une documentation complète permettant l'ajout de nouvelles bases de données sur le long terme

3 Analyse des données

3.1 Présentation des données

Le projet repose sur un ensemble de 22 bases de données de polypes colorectaux présentant des structures, formats et conditions d'acquisition fortement hétérogènes. L'objectif est de les intégrer dans une représentation unifiée afin de permettre la génération de jeux de données exploitables pour l'apprentissage automatique. Dans l'état actuel, l'exploitation conjointe de ces bases est complexe. En effet, les informations nécessaires à la constitution de datasets (labels, annotations, conditions d'acquisition ou métadonnées) sont dispersées, non standardisées ou parfois absentes, ce qui empêche toute interrogation multi-base cohérente.

3.2 Hétérogénéité des données

L'analyse des bases de données met en évidence plusieurs niveaux d'hétérogénéité limitant leur exploitation conjointe.

Hétérogénéité des conditions d'acquisition Les images présentent des variations importantes liées aux conditions d'imagerie : type de caméra, résolution, éclairage, présence ou absence de colorants. De plus, ces informations ne sont pas toujours disponibles selon la base de données. Par exemple, sur l'image de droite ci-dessous, on observe l'utilisation d'un colorant. Mais cette information n'existe que visuellement. Par ailleurs, ces trois images issues de bases de données différentes ont toutes une résolution différente. Et sur l'image de droite, la lumière utilisée n'est pas la même que pour les deux autres.



Figure 1: Exemple de variabilité des conditions d'acquisition : différences de résolution, d'éclairage, et utilisation de colorants.

Hétérogénéité des annotations (bounding boxes) Les annotations associées aux images présentent également une forte variabilité. Selon les bases de données, les objets d'intérêt peuvent être annotés sous forme de bounding boxes, mais avec des formats différents (TXT, XML, CSV) et des conventions non uniformes. Par ailleurs, il n'y a pas de façon uniformisée de représenter une zone d'intérêt : elle peut être présente ou non sur l'image.



Coordonnées de la bbox contenues dans un fichier csv : 23,0,298,335

Coordonnées de la bbox contenues dans un fichier xml
<xmin>177</xmin>
<ymin> 181</ymin>
<xmax> 262</xmax>
<ymax> 241</ymax>

Coordonnées de la bbox contenues dans un fichier txt
24 544 872 704

Figure 2: Exemples de représentations des annotations de type bounding boxes et diversité des formats associés (TXT, XML, CSV).

Hétérogénéité des segmentations Les segmentations constituent un autre niveau d'hétérogénéité. Elles peuvent prendre différentes formes selon les datasets : masques binaires, contours uniquement, ou encore segmentations colorées où la couleur encode parfois une information sémantique supplémentaire comme le type de diagnostic. Par exemple, les versions segmentées des images de la base de données BKAI-IGH NeoPolyp sont rouges pour les polypes néoplasiques (lésion tumorale ou à potentiel tumoral) et vertes pour les polypes non-néoplasiques.

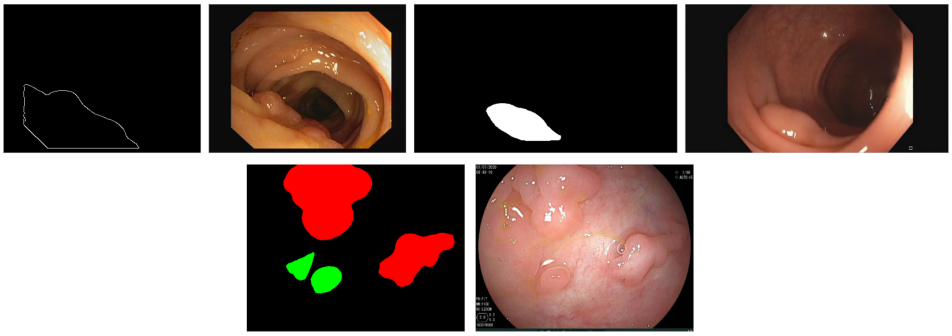


Figure 3: Exemples d'images accompagnées de leur version segmentée

Hétérogénéité des systèmes de labellisation Les bases de données utilisent différents systèmes de labellisation et de classification (CONECCT, Kudo, Paris, Nice, Jnet, LST). De plus, un même diagnostic peut être représenté par des labels différents et/ou avec des niveaux de granularité variables.
Certaines bases de données, comme AIDA-E, ne distinguent que deux classes générales telles que "Normal" et "Abnormal". D'autres différencient uniquement la présence ou l'absence de polypes. Enfin, certaines bases proposent une classification plus fine, distinguant par exemple plusieurs types de polypes tels que les adénomes, les lésions hyperplasiques ou les polypes sessiles festonnés.
Certaines bases fournissent également un diagnostic plus détaillé selon une ou plusieurs classifications médicales spécifiques.
Cela suggère qu'il faut créer un schéma de représentation des diagnostics unifié pour garantir la cohérence des données.

Manque de métadonnées Enfin, plusieurs métadonnées essentielles sont absentes pour une partie des bases de données, notamment :

- l'origine des données (hôpital, pays, patient),
- les conditions d'acquisition (type de caméra et paramètres associés),
- les informations relatives aux annotateurs (expertise, spécialité, niveau d'expérience).

Ces informations sont pourtant importantes, car elles influencent la variabilité des annotations et la robustesse des modèles d'apprentissage.
Une synthèse détaillée des bases de données étudiées est fournie en annexe, incluant leur structure, leurs formats de labels et les métadonnées disponibles ou manquantes.

4 Méthode : choix conceptuels

4.1 Solutions explorées

Nous avons exploré différents modèles de représentation des informations: Le modèle relationnel, document et graphe RDF[3]. Au fil de notre étude, nous avons pu établir un cahier des charges auquel le modèle doit pouvoir répondre:

Option	Forme	Avantages	Inconvénients
Modèle Relationnel	Tableau: ID | Dataset | FORMAT img_001 | DB_01 | "jpg"	• Lecture simplifiée • Solution locale	• Pas de notions de hierarchie native • Schéma rigide • Relation des classes avec jointure : création de plusieurs tableaux
Modèle Document	JSON: { "image_id": "img_001", "diagnosis": { "annotator" = "an_01",..} "bbox": {...}, "dataset": "db_01"}	• Relation entre les classes • Schéma flexible • Solution locale	• Répétition de l'information • Hiérarchie sans inférence
Modèle Graphe	Graphe RDF: Image —hasDiagnosis→ Diagnostic	• Gestion de la hiérarchie et des classes • Flexibilité • Gestion des données manquantes • Solution locale	• Courbe d'apprentissage du langage • Lecture difficile

Figure 4: Tableau de comparaison des modèles envisagés

- **Solution locale :** Certaines données étant protégées et pour éviter l'accès aux informations et recherches par des acteurs privés, nous souhaitons des modèles implémentables par outils de traitement de requêtes indépendants et locaux. Nous avons trouvé SQLite[3] pour le modèle relationnel, le langage de requêtes N1QL [5] sur un serveur local pour le modèle document et une interface locale pour des requêtes SPARQL du modèle graphe.
- **Représentation des entités** : Le modèle doit pouvoir représenter les littéraux liés à une entité, comme par exemple le format d'une image sous forme de caractère. Le modèle tabulaire réponds à la problématique en représentant les attributs de l'entité sous forme de n-uplets. Le modèle document associe le nom de l'attribut et sa valeur tandis que le modèle graphe relie l'entité et son attribut sous forme de triplets.
- **Relation entre les entités** : Au delà des littéraux, certaines classes sont reliées comme le diagnostic d'une image ou la base de donnée initiale d'une image. La solution proposée doit donc pouvoir permettre de retrouver les informations externes d'une image. Le modèle relationnel se résume à retrouver l'entité externe à l'aide d'une clé étrangère. Cela nécessite de créer et gérer plusieurs tables d'entités comme `Diagnostic`. Le modèle document utilise une structure imbriquée : Les entitées liées sont encapsulées, par exemple le diagnostic d'une image est contenue dans la définition de celle-ci. Toutefois, cela implique une mise-à-jour des informations plus diffile car il faut parcourir toutes les images et effectuer une requête double pour la gestion des clés étrangères comme le modèle relationnel. Le modèle graphe RDF peut simplement créer une propriété entre les deux classes.

- **Extensibilité :** Le modèle doit assurer la pérennité et l'évolution du modèle face aux changements futurs. Cela passe par un ajout simplifié de nouvelles images ou de nouveaux attributs introduits par une nouvelle base de données. Le modèle relationnel est peu adapté car il necessite une structure rigide, c'est à dire une définition préalable des attributs existants et leur type. La modification fréquentes des tables peut entraîner des incohérences et erreurs. Les modèles document et graphe RDF sont flexibles car ils ne nécessitent pas de squelette fixe.
- **Gestion de la hiérarchie :** Il existe différents niveaux de précision pour un diagnostic : Un polype peut être un adenome et l'adenome peut être un adénome tubulaire. Le modèle choisit doit pouvoir retrouver les différents niveaux de diagnostic pour avoir un résultat complet lors des requêtes. Le modèle document n'a pas d'outil de gestion de la hiérarchie et ne peut pas faire d'inférence, il faudrait répéter l'information à chaque niveau. Contrairement au modèle document, le modèle graphe RDF à un outil d'inférence avec la propriété `rdfs:subClassOf`.

4.2 Solution retenue : modèle graphe RDF

Le modèle le plus adapté à notre contexte médicale est le modèle graphe RDF grâce à sa flexibilité, la gestion des classes et son extensibilité. Il nous permet de représenter les informations liées aux images par un graphe de connaissances.

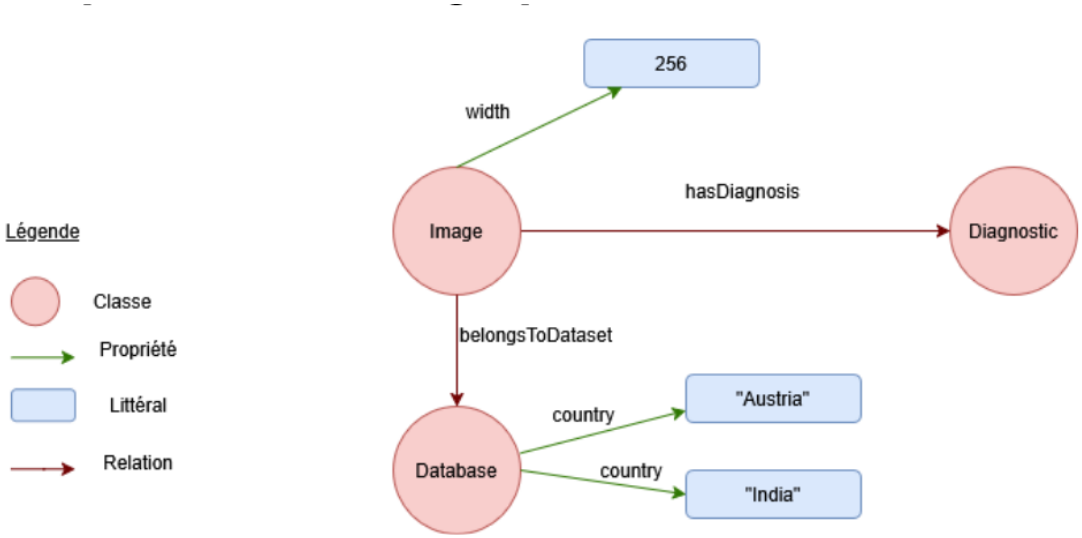


Figure 5: Extrait du graphe de connaissance RDF

Maintenant que nous nous somme fixés sur le modèle, nous devons établir une ontologie, c'est-à-dire créer un répértoire des classes, sous-classes, propriétés et relations existantes. Les propriétés sont les informations primaires d'une classe (format d'une image, nom d'une base de donnée...) tandis que les relations relient une classe à une autre mais relient également les mères et filles.

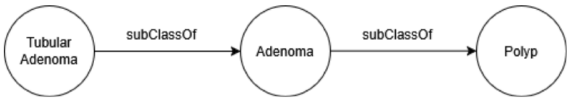


Figure 6: Schéma des relations entre une classe mère et une classe fille

Les informations de l'ontologie sont indispensables à repérer pour chaque parseur, mais également pour l'écriture de requêtes valides.

Un espace de noms personnalisé a été défini : http://example.org/polyps (associé au préfixe EX:) qui représente l'univers du graphe de connaissance du projet : toutes les définitions sont representé dans cet espace. Chaque entité (une image originale, un masque de segmentation, un polype ou un diagnostic) se voit attribuer une URI unique construite dynamiquement lors de chaque parsing. Par exemple,

une image provenant du dataset CVC-ClinicDB sera identifiée par une URI propre et unique, tandis que ses relations (comme le lien vers son polype ou son type de segmentation) utiliseront des prédicats sémantiques stricts issus soit de l'ontologie native du projet, soit de la sémantique RDF (comme rdf:type).

L'implémentation du graphe de connaissances a été réalisée en Python à l'aide de la bibliothèque de référence `Rdflib`. Le script est structuré autour d'un graphe globalisé (`g = Graph()`) chargé d'accumuler l'ensemble des triplets de chaque base de donnée.

5 Implémentation de la solution

5.1 Pipeline : Méthode de construction de parseurs

Bien que l'on puisse voir des motifs qui se répètent, chaque base de données possède une structure unique qu'il faut étudier rendant un modèle d'implémentation unique impossible. Néanmoins, nous proposons une méthode pour une implémentation réussie d'une nouvelle base:

1. **Parcours des images et extractions des métadonnées à partir de fichiers annexes :** La première étap est primordiale et consiste à étudier où se trouve l'information de chaque base de données : certaines contiennent uniquement le diagnostic dans le nom de fichier de l'image tandis que d'autres possèdent des fichiers annexes complets (CSV, JSON, EXCEL, XML...) avec des annotations humaines comme les informations sur les patients, les différents diagnostics (Plusieurs avis médicaux sur l'image) ou encore sur la localisation du polype (coordonnées). Il faut donc implémenter les fonctions de lecture en accord avec le fichier et éviter la perte d'information.

2. **Normalisation des terminologies :** Il n'y a pas d'écriture standard des diagnostics à travers les bases de données. Un même diagnostic peut avoir des écritures différentes comme par exemple SessileSerrated qui peut se décliner en sessile, sessileserrated ou serrated. L'étape de normalisation consiste à convertir les données écrites en langage naturelles, en données URI propres, uniques et unifiées pour toutes les bases de données à l'aide de dictionnaires.

3. **Génération des triplets RDF et intégration dans le graphe de connaissance :** Après avoir détécter toutes les données et les avoir rendu propre, on les intègre dans le graphe de connaissance par des ajouts multiples sous forme de triplets. Le graphe final consistera en une liste de triplets avec des attributs et prédicats unifiés de chaque base de données.

5.2 Extensbilité

Pour rendre la construction de parseurs accessible et assurer que le modèle de données soit réellement pérenne, nous nous comme assurés que le code reste le plus transparent et réutilisable possible. Pour cela, plusieurs outils sont mis à disposition pour les implémentations ou modifications futures.

Librairie : Des fonctions de générations de triplets ont été mise en place pour les triplets redondants comme par exemple `create_image` qui génère les triplets de données techniques d'une image (dimensions, nom, durée si vidéo..) ainsi que des triplets de relations reliant une image à son diagnostic ou encore une image et le polype qu'elle représente lorsque celui ci est partagé par plusieurs images. La librairie offre également des dictionnaires de conversion pour permettre le nettoyage des données et obtenir les URIs existants des diagnostics. Par la suite, il sera possible d'ajouter de nouveaux noms de diagnostics ou modifier ceux existants pour que les anciennes bases de données s'adaptent rétroactivement.

Documentation : Un guide méthodologique est mis à disposition pour rendre le projet le plus accessible possible. D'une part, la description de la librairie avec les différentes fonctions annexes y est présente mais également l'architecture complète du code principale y est décrite pour mieux comprendre la strucuture d'une projet. Le guide indique les bonnes pratiques à avoir et comment atteindre une bonne implémentation (Comment parcourir efficacement les données, les premiers pas à avoir..) mais également les questions et enjeux auxquels il faut savoir répondre à chaque étape de l'implémentation comme "Ou se trouve le diagnostic ? Est-ce que la base de donnée possède plusieurs annotateurs? Plusieurs classifications utilisées? ". Ces questions permettent d'éviter la perte d'informations lors d'une nouvelle implémentation. Un tableau récapitulatif de toutes les bases de données est inscrit dans le guide pour pouvoir s'inspirer de plusieurs parseurs qui se ressemblent sur la structure. La documentation a également

une section exploration de données offrant une liste exhaustive de l'ontologie pour aider lors de la phase interrogation de donnée.

5.3 Interface d'interrogation

Le requêtage d'un graphe RDF stocké au format Turtle s'effectue via le langage SPARQL. Cependant, la courbe d'apprentissage de ce langage peut constituer un frein pour des utilisateurs non experts, notamment dans un contexte médical ou de recherche appliquée. Afin de faciliter l'accès aux données, nous avons développé une interface permettant d'interroger la base de manière intuitive à travers un système de filtres.
Cette interface permet notamment :

- la récupération d'ensembles d'images à partir de plusieurs datasets avec leurs métadonnées associées pour l'apprentissage automatique,
- la sélection d'images selon des critères cliniques ou techniques (type de classification, base de données d'origine, annotations),
- l'exploration de la distribution des données selon différents attributs (datasets, labels, conditions d'acquisition).

5.3.1 Structure

L'interface est composée de deux pages principales :

- Query Builder : génération automatique de requêtes SPARQL à partir de filtres utilisateurs.
- Data Explore : visualisation de statistiques globales sur la base de données (distribution des datasets, répartition des labels, volume d'images par source, etc.)

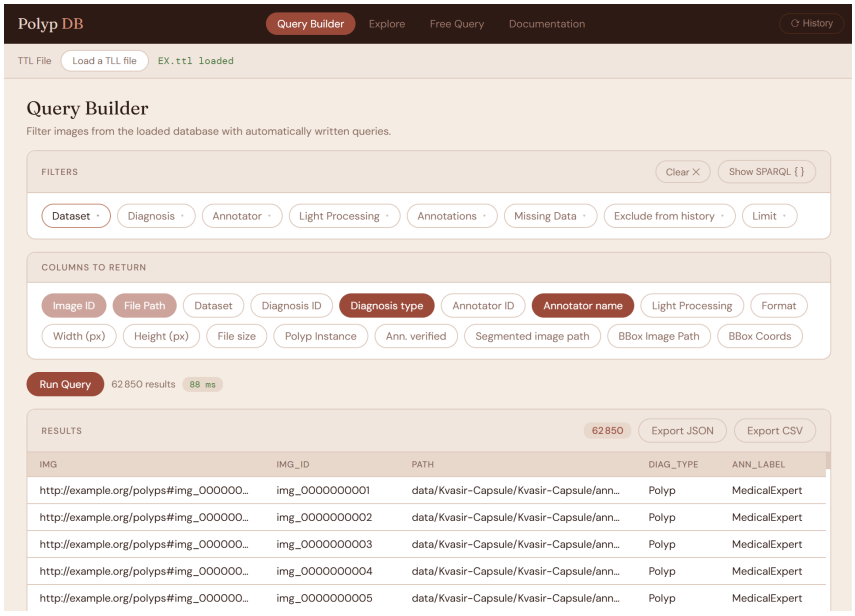


Figure 7: Page Query Builder

Il existe deux autres pages : la page Free Query qui permet d'écrire des requêtes libres ainsi que la page de documentation qui explique en anglais le fonctionnement de l'interface.
L'interface est développée en anglais afin de garantir l'accessibilité à un plus grand nombre.

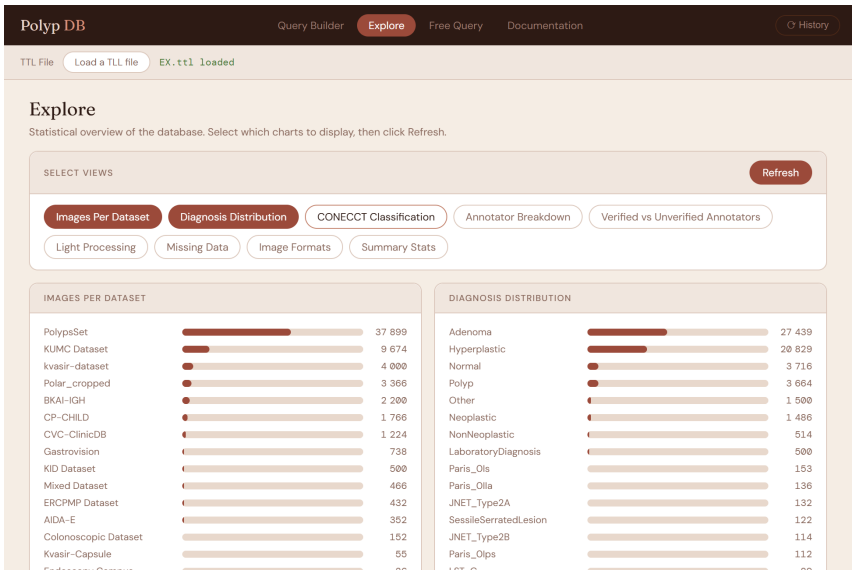


Figure 8: Page Explore

5.3.2 Architecture et optimisation des performances

L'interface repose sur des technologies web standard (HTML, CSS et JavaScript). Le requêtage RDF est assuré par les bibliothèques N3.js et Comunica, permettant l'exécution de requêtes SPARQL sur des fichiers Turtle en environnement local.
Afin d'optimiser les performances, nous avons mis en place un système d'indexation construit lors du chargement du fichier RDF. Cet index repose sur des structures de type dictionnaire permettant un accès direct aux entités les plus fréquemment requêtées. Grâce à lui, on ne lit pas le million de triplets présents dans la base. Tous les types, propriétés, classes et relations présents dans les filtres sont indexés. Ainsi, les filtres sélectionnés par l'utilisateur sont d'abord traduits en requête interne exploitant l'index. Les résultats intermédiaires sont ensuite, si nécessaire, traités par le moteur SPARQL pour garantir l'expressivité complète du langage.
Les performances observées montrent un gain extrêmement significatif :

- avec index : 50 à 370 ms selon la complexité des requêtes,
- sans index : jusqu'à plusieurs dizaines de minutes pour des requêtes volumineuses.

L'architecture hybride de l'interface permet de combiner rapidité d'exécution et flexibilité des requêtes, tout en rendant l'interrogation de la base accessible à des utilisateurs non experts. La documentation complète est en annexe. Elle assure sa pérennité en décrivant comment ajouter des filtres en cas d'ajouts de nouvelles bases.

6 Limitations

Bien que le système proposé permette d'unifier, d'interroger et d'exploiter les données médicales hétérogènes, plusieurs limitations subsistent.

Gestion de bases de données volumineuses L'intégration de nombreuses bases de données peut entraîner des limitations en termes de stockage. Certaines bases de données, du fait du nombre d'images disponibles (5 millions dans le cas de la base de données GastroNet), sont très longues à parser. Une amélioration possible serait de mettre en place un traitement incrémental des données qui parallélise le processus de parsing par batch.

Déduplication des données Certaines bases de données sont des agrègent plusieurs autres bases de données existantes. C'est le cas de Mixed Dataset qui crée donc des doublons dans notre base unifiée. D'autres cas concernent des quasi-doublons, en particulier lorsque les images sont des captures d'écran de vidéos très rapprochés et présentent de très fortes similarités sans être identiques.

La distinction entre ces deux cas rend la mise en place d'un mécanisme de déduplication compliquée. Une approche possible est la représentaion desimages sous formes d'embeddings extraits à l'aide de réseaux de neurones pré-entraînés comme exemple ResNet, puis à utiliser une méthode de recherche de similarité telle que FAISS.

Détection et gestion de bruit Certaines images contiennent contiennent du bruit ce qui peut affecter la robustesse des modèles d'apprentissage. Certains éléments sont facilement identifiables visuellement (présence d'outils chirurgicaux, zones masquées, éléments graphiques superposés), mais ne sont pas explicitement annotés dans les bases de données.



Figure 9: Exemples d'images bruitées provenant de trois datasets différents

Une perspective d'amélioration consiste à intégrer un module de filtrage automatique basé sur des modèles pré-entraînés capables de détecter des artefacts pour annoter ou écarter certaines observations.

Absence d'équivalence entre les classifications médicales Une limitation importante concerne l'absence de correspondance exacte entre les différentes classifications médicales utilisées dans les bases de données (notamment CONECCT, Kudo et la classification de Paris). En effet, ces systèmes reposent sur des critères différents, par exemple des critères morphologiques pour la classification de Paris et des critères d'architecture de surface pour la classification de Kudo. Cette absence de correspondance univoque complique l'harmonisation automatique des labels.

7 Conclusion

Ce projet apporte une solution pérenne au problème des images médicales stockées dans des dossiers de structures toutes différentes. En rassemblant tous ces fichiers dans un unique graphe unifié de données, nous avons créé un système structuré mais très flexible grâce au modèle RDF : ajouter une nouvelle base se fait désormais à l'aide d'un guide méthodologique d'intégration de donnée complet.
L'intérêt principal de cette solution est qu'elle s'adapte à tout le monde. Les utilisateurs experts peuvent faire des recherches précises directement avec des requêtes SPARQL, tandis que les non-experts peuvent interroger l'ensemble des images et des diagnostics à l'aide d'une interface intuituive.

8 Bibliographie

[1] Institut National du CAncer. Panorama des cancers en france 2025. Technical report, 2025.
2 . Clementine Brule et al. The colorectal neoplasia endoscopic classification to choose the treatment classification for identification of large laterally spreading lesions lacking submucosal carcinomas : A prospective study of 663 lesions. *United European gastroenterology journal*, 10(1) :80–92, 2022.
3 . Richard Cyganiak, David Wood, and Markus Lanthaler. RDF 1.1 Concepts and Abstract Syntax. *W3C Recommendation*, 2014. https://www.w3.org/TR/rdf11-concepts/

4 . D. Richard Hipp and SQLite Development Team. SQLite Storage Engine and Database Management System. *SQLite Consortium*, 2026. Disponible sur https://sqlite.org/. .
5. Gerald Sangraft and Couchbase Development Team. N1QL: A Language for Querying JSON Documents. *Couchbase, Inc.*, 2026. Disponible sur https://www.couchbase.com/products/n1ql.